MODULE 1

Introduction to Machine Learning

Foundations and Data Preprocessing

Complete Study Notes

From first principles to professional practice

Course instructor
Prof. Rajesh Nag

Topics covered

Meaning and scope of ML • Types of learning • Applications • ML workflow • Data types • Preprocessing • Handling missing values • Feature selection • Training and testing data

Worked examples, diagrams, code, exam tips and a revision toolkit included.

How to Use These Notes

These notes are written for **every kind of reader**: if you are new to the subject, start with the green *In Plain Words* boxes; if you are preparing for tests, focus on definitions, tables and worked examples; if you already work with data, the purple *Pro Corner* boxes add depth.

Box	What it gives you
In Plain Words	Everyday analogy; the idea with no jargon
Definition / Key Idea	Exact wording to write in exams
Worked Example	Step-by-step numerical or practical example
Tip	Helpful shortcuts and intuition
Watch Out	Common mistakes and traps
Exam Tip	How questions are usually asked and answered
Pro Corner	Deeper, professional-level details

Suggested study plan: read a chapter once for understanding, redo every worked example without looking, then attempt the practice questions in Chapter 10. Chapters 6–9 (preprocessing, missing values, feature selection, splitting) are where numerical questions most often appear.

Contents

How to Use These Notes i

1 Meaning and Scope of Machine Learning 1
- 1.1 What is Machine Learning? . . . 1
- 1.2 Traditional Programming vs. Machine Learning . . . 1
- 1.3 AI, Machine Learning, Deep Learning and Data Science . . . 2
- 1.4 Why Do We Need Machine Learning? . . . 3
- 1.5 A Short History of ML . . . 3
- 1.6 Scope of Machine Learning . . . 3
- 1.7 Essential Vocabulary . . . 4

2 Types of Learning 5
- 2.1 Supervised Learning . . . 5
- 2.2 Unsupervised Learning . . . 6
- 2.3 Semi-Supervised Learning . . . 7
- 2.4 Self-Supervised Learning . . . 7
- 2.5 Reinforcement Learning (RL) . . . 7
- 2.6 Comparison of the Learning Types . . . 9
- 2.7 Other Ways to Classify Learning Systems . . . 9

3 Applications of Machine Learning 10
- 3.1 Four Mini Case Studies . . . 10
- 3.2 Responsible Use: Ethical Issues . . . 11

4 The Machine Learning Workflow 12
- 4.1 Step-by-Step Explanation . . . 12
- 4.2 Evaluation Basics You Must Know . . . 13
- 4.3 Overfitting, Underfitting and the Bias–Variance Trade-off . . . 14
- 4.4 Putting the Workflow in Code . . . 14

5 Data and Its Types **16**
5.1 Anatomy of a Dataset . . . 16
5.2 Classification by Structure . . . 16
5.3 Classification by the Nature of the Attribute . . . 16
5.4 Stevens' Four Levels of Measurement . . . 17
5.5 Other Kinds of Data You Will Meet . . . 18
5.6 Data Formats and Sources . . . 18
5.7 Data Quality and Big Data . . . 18
5.8 Why Data Type Drives Everything Next . . . 18

6 Data Preprocessing **20**
6.1 Data Cleaning . . . 20
6.2 Data Integration . . . 21
6.3 Data Transformation . . . 21
6.4 Data Reduction . . . 23
6.5 Handling Imbalanced Data . . . 23
6.6 Preventing Data Leakage with Pipelines . . . 23
6.7 Preprocessing Checklist . . . 23

7 Handling Missing Values **25**
7.1 Why Are Values Missing? . . . 25
7.2 Types of Missingness . . . 25
7.3 Step 1: Detect and Understand . . . 25
7.4 Step 2: Choose a Strategy . . . 25
7.5 Decision Guide . . . 27
7.6 Code . . . 27

8 Feature Selection **29**
8.1 Why Select Features? . . . 29
8.2 Feature Selection vs. Extraction vs. Engineering . . . 29
8.3 Three Families of Selection Methods . . . 30
8.4 Filter Methods . . . 30
8.5 Wrapper Methods . . . 31
8.6 Embedded Methods . . . 31
8.7 Feature Extraction: a First Look at PCA . . . 32
8.8 Practical Code . . . 32

9 Training and Testing Data **34**
9.1 The Three Datasets . . . 34
9.2 Splitting Strategies . . . 34
9.3 Diagnosing Models from Train and Test Scores . . . 36
9.4 Data Leakage: The Silent Killer . . . 36
9.5 Code . . . 36

10 Revision Toolkit **38**
10.1 One-Page Summary . . . 38
10.2 Formula Sheet . . . 38
10.3 Commonly Confused Pairs . . . 39
10.4 Short-Answer Practice (with model answers) . . . 39
10.5 Long-Answer Outlines . . . 40
10.6 Multiple-Choice Practice . . . 40
10.7 Glossary . . . 41

1 Meaning and Scope of Machine Learning

1.1 What is Machine Learning?

In Plain Words

Imagine teaching a child to recognise a dog. You do not hand over a rulebook saying "four legs, fur, tail, barks". You simply point at many dogs and say "dog", and at cats and say "not a dog". After enough examples the child spots a dog they have never seen before. **Machine Learning (ML) is the same idea for computers:** instead of writing rules by hand, we show the computer many examples and let it discover the rules itself.

Machine Learning is a branch of Artificial Intelligence in which computers **learn patterns from data** and use those patterns to make predictions or decisions on new, unseen data, without being explicitly programmed for every situation.

Definition: Machine Learning (Arthur Samuel, 1959)

The field of study that gives computers the ability to learn without being explicitly programmed.

Definition: Machine Learning (Tom Mitchell, 1997) — the formal, exam-favourite definition

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Worked Example: Applying Mitchell's definition to a spam filter

- **Task T:** classify incoming emails as spam or not spam.
- **Experience E:** a collection of emails already labelled by users as spam or not spam.
- **Performance P:** the percentage of emails classified correctly (accuracy).

If accuracy goes up as the filter sees more labelled emails, the program is *learning*.

Exam Tip

Whenever a question says "define machine learning", write *both* definitions (Samuel's and Mitchell's) and then give the spam-filter $T/E/P$ example. This alone usually secures full marks for the definition part.

1.2 Traditional Programming vs. Machine Learning

In **traditional programming** a human writes the rules; the computer applies them to data to produce answers. In **machine learning** the human supplies data *and* the correct answers; the computer works out the rules (called the **model**).

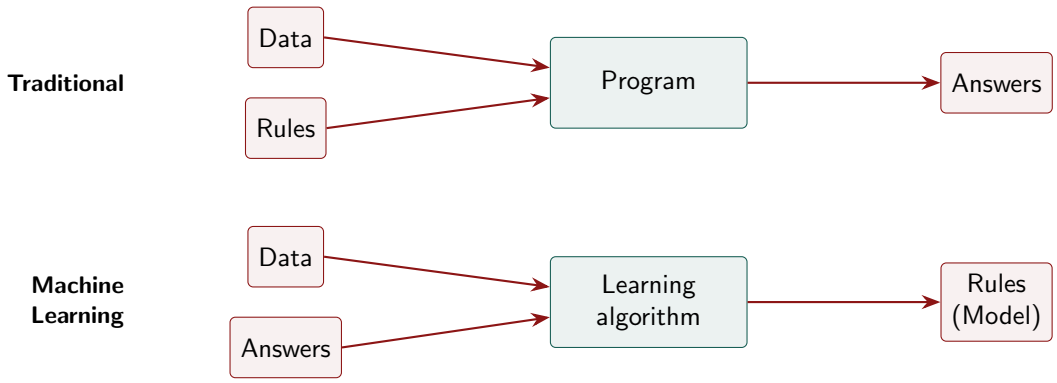


Figure 1. Traditional programming (rules in, answers out) versus ML (answers in, rules out).

Aspect	Traditional programming	Machine learning
Input	Data + hand-written rules	Data + known answers (usually)
Output	Answers	Rules (a trained model)
Who finds the logic?	The programmer	The algorithm
Handles change?	Programmer must edit the code	Retrain with new data
Best suited for	Clear, fixed, deterministic logic (tax calculation, sorting)	Fuzzy patterns (faces, speech, fraud, recommendations)
Quality depends on	Correctness of the code	Quality and quantity of data

1.3 AI, Machine Learning, Deep Learning and Data Science

These four terms are often mixed up. Think of them as *nested circles*.

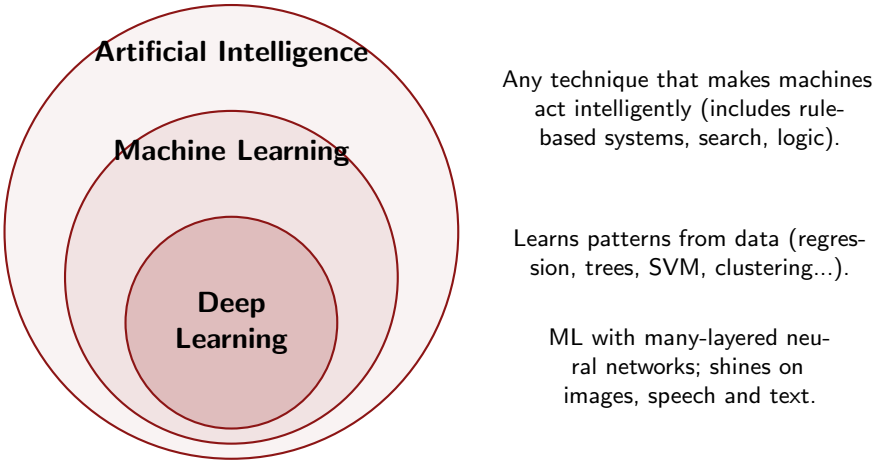


Figure 2. Deep Learning ⊂ Machine Learning ⊂ Artificial Intelligence.

Term	Meaning	Example
Artificial Intelligence	Building systems that perform tasks needing human-like intelligence	A chess engine using hand-coded search and evaluation rules
Machine Learning	AI systems that improve from data	Email spam filter
Deep Learning	ML using deep neural networks	Face unlock on a phone, speech assistants
Data Science	Extracting insight from data using statistics, programming and domain knowledge; ML is one of its tools	Analysing sales data to decide where to open a new store

1.4 Why Do We Need Machine Learning?

ML is the right tool in these situations:

1. **The rules are too complex to write.** Nobody can hand-code every rule for recognising handwriting.
2. **The rules keep changing.** Spammers change tactics daily; a learned filter can be retrained.
3. **Humans cannot see the pattern** because the data is huge or high-dimensional (millions of transactions, thousands of genes).
4. **Personalisation at scale.** Different recommendations for millions of users.
5. **Learning from experience improves results** over time (self-driving, game-playing).

Watch Out

ML is **not** always the answer. Avoid it when: (i) a simple exact rule exists ("if age $\geq$ 18 then adult"), (ii) you have very little data, (iii) there is no real pattern in the data, or (iv) errors are unacceptable and you need a guaranteed, fully explainable decision.

1.5 A Short History of ML

Year	Milestone
1943	McCulloch and Pitts propose a mathematical model of a neuron
1950	Alan Turing publishes "Computing Machinery and Intelligence" (the Turing Test)
1952	Arthur Samuel's checkers program learns to improve by playing
1957	Frank Rosenblatt invents the Perceptron
1967	Nearest-neighbour algorithm (Cover and Hart)
1986	Backpropagation popularised (Rumelhart, Hinton, Williams)
1995	Support Vector Machines (Cortes and Vapnik)
1997	IBM Deep Blue defeats chess champion Garry Kasparov
2006	Hinton and colleagues revive deep neural networks
2012	AlexNet wins ImageNet competition by a huge margin
2017	Transformer architecture introduced
2020s	Large language models and generative AI go mainstream

1.6 Scope of Machine Learning

Where ML is used today: healthcare, finance, e-commerce, transport, agriculture, education, manufacturing, cyber-security, entertainment and government services (details in Chapter 3).

Major sub-areas:

- **Computer Vision** — understanding images and video.
- **Natural Language Processing (NLP)** — understanding and generating human language.
- **Speech Recognition and Synthesis.**
- **Recommender Systems** — suggesting products, movies, songs.
- **Time-series Forecasting** — sales, stock, weather and demand prediction.
- **Robotics and Control** — learning to act in the physical world.

- **Anomaly Detection** — fraud, faults, intrusions.

Careers: Data Analyst, Data Scientist, ML Engineer, AI Researcher, MLOps Engineer, NLP Engineer, Computer Vision Engineer.

Limitations and challenges

- **Data hunger and quality:** poor or biased data gives poor or biased models ("garbage in, garbage out").
- **Overfitting:** the model memorises training data and fails on new data.
- **Interpretability:** complex models behave like black boxes.
- **Bias and fairness:** models can inherit unfair patterns from historical data.
- **Privacy and security:** personal data, adversarial attacks.
- **Concept drift:** the world changes, so yesterday's model decays.
- **Computational cost:** training large models needs heavy hardware.

1.7 Essential Vocabulary

Term	Meaning
Sample / instance / record / row	One example in the dataset (one email, one patient, one house)
Feature / attribute / variable / column	A measurable property of a sample (age, area in sq. ft., word count)
Label / target / response	The answer we want to predict (spam or not, house price)
Dataset	Collection of samples
Model	The learned mathematical function that maps features to predictions
Parameter	Value learned *from data* during training (e.g. weights in regression)
Hyperparameter	Setting chosen *by us before* training (e.g. k in k-NN, learning rate)
Training	The process of fitting the model to data
Inference / prediction	Using the trained model on new data
Generalisation	Performing well on unseen data (the real goal of ML)

Pro Corner (for professionals)

Formal view. Assume data are drawn from an unknown distribution $\mathcal{D}$ over pairs (x, y). We search a *hypothesis space* $\mathcal{H}$ for a function f minimising the true risk $R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[L(f(x), y)]$. Since $\mathcal{D}$ is unknown, we minimise the *empirical risk* on n training points,

$$\hat{R}(f) = \frac{1}{n}\sum_{i=1}^{n} L\big(f(x_i), y_i\big),$$

(Empirical Risk Minimisation). The gap between $\hat{R}(f)$ and $R(f)$ is the generalisation gap. The **No Free Lunch theorem** states that no single learning algorithm is best across all possible problems; every algorithm encodes an **inductive bias** (assumptions) that suits some problems and not others.

2 Types of Learning

ML algorithms are grouped by **how much and what kind of supervision** they get during training.

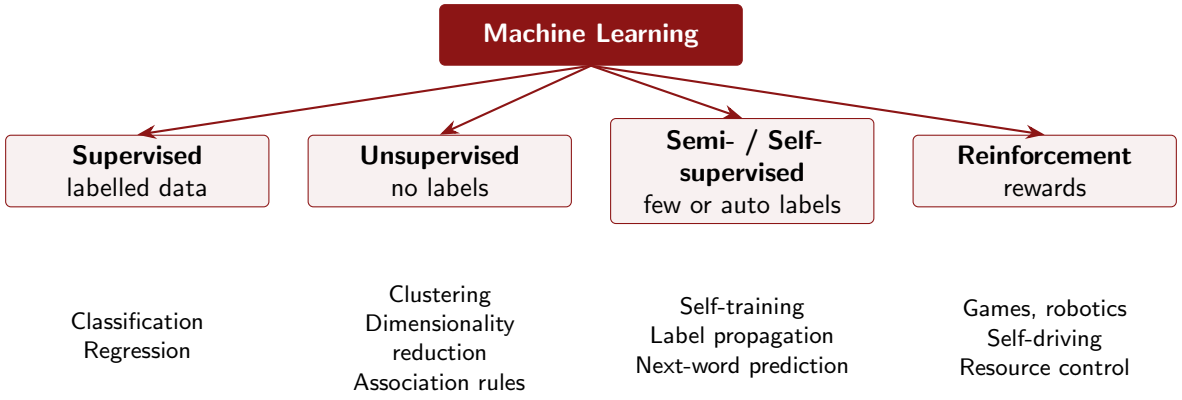


Figure 3. The main families of machine learning.

2.1 Supervised Learning

In Plain Words

A teacher shows you a question *and* its correct answer, again and again. Later you face new questions alone. That is supervised learning: every training example comes with the right answer (the *label*).

Definition: Supervised Learning

Learning a function $f : X \to Y$ from a training set of labelled pairs $\{(x_1, y_1), \ldots, (x_n, y_n)\}$ so that $f(x)$ predicts y accurately for new, unseen x.

There are two kinds of supervised problems, decided by the type of the label y:

	Classification	Regression
Output y	Discrete category (class)	Continuous number
Question answered	"Which one?"	"How much?"
Examples	Spam / not spam; disease / healthy; digit 0–9	House price; temperature tomorrow; exam score
Sub-types	Binary, multiclass, multilabel	Simple, multiple, polynomial
Typical algorithms	Logistic regression, k-NN, decision tree, SVM, Naive Bayes	Linear regression, ridge, lasso, regression trees
Typical metrics	Accuracy, precision, recall, F1	MAE, MSE, RMSE, R^2

Popular supervised algorithms: Linear Regression, Logistic Regression, k-Nearest Neighbours, Decision Trees, Random Forest, Support Vector Machines, Naive Bayes, Gradient Boosting, Neural Networks.

How does a supervised model actually learn?

1. Start with a model with random or zero parameters.
2. Make predictions $\hat{y}$ on training data.
3. Measure the error with a **loss function**, e.g. Mean Squared Error: $J = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$.

4. Adjust parameters to reduce the loss using an **optimiser** such as gradient descent: $\theta \leftarrow \theta - \alpha \nabla_\theta J(\theta)$, where α is the learning rate.
5. Repeat until the loss stops improving.

Worked Example: One gradient-descent step for $\hat{y} = wx + b$

Data: $(1, 2)$ and $(2, 4)$. Start $w = 0,\ b = 0$, learning rate $\alpha = 0.1$, $J = \frac{1}{n}\sum(\hat{y} - y)^2$, so $\frac{\partial J}{\partial w} = \frac{2}{n}\sum(\hat{y} - y)x$ and $\frac{\partial J}{\partial b} = \frac{2}{n}\sum(\hat{y} - y)$.

Before: predictions are $0, 0$; errors $(\hat{y} - y) = -2, -4$; $J = \frac{4+16}{2} = 10$.
$\frac{\partial J}{\partial w} = \frac{2}{2}\big[(-2)(1) + (-4)(2)\big] = -10, \quad \frac{\partial J}{\partial b} = \frac{2}{2}\big[-2 - 4\big] = -6.$
Update: $w = 0 - 0.1(-10) = 1.0, \quad b = 0 - 0.1(-6) = 0.6.$
After: predictions are 1.6 and 2.6; $J = \frac{(1.6-2)^2+(2.6-4)^2}{2} = \frac{0.16+1.96}{2} = 1.06.$
The loss fell from 10 to 1.06 in a single step. Repeating this many times converges to $w = 2,\ b = 0$.

2.2 Unsupervised Learning

In Plain Words

You are handed a huge box of mixed buttons with no instructions. You naturally sort them into piles by colour, size or shape. Nobody told you the "right" groups; you found structure on your own. That is unsupervised learning.

Definition: Unsupervised Learning

Learning patterns, structure or relationships from data that has **no labels**. The algorithm explores the data and organises it by itself.

Main tasks

- **Clustering:** group similar samples (customer segmentation). Algorithms: k-Means, Hierarchical, DBSCAN.
- **Dimensionality reduction:** compress many features into fewer while keeping information (PCA, t-SNE, autoencoders).
- **Association rule mining:** find items that occur together (market-basket analysis). Algorithms: Apriori, FP-Growth.
- **Anomaly detection:** find rare, unusual samples (Isolation Forest, one-class SVM).

k-Means in five steps

1. Choose k (number of clusters).
2. Place k centroids (randomly).
3. **Assign** each point to its nearest centroid.
4. **Update** each centroid to the mean of its assigned points.
5. Repeat steps 3–4 until assignments stop changing.

Worked Example: k-Means on 1-D data

Points: $1, 2, 3, 10, 11, 12$; $k = 2$; initial centroids $c_1 = 1,\ c_2 = 10$.
Iteration 1, assign: $1, 2, 3$ are closer to c_1; $10, 11, 12$ are closer to c_2. **Update:** $c_1 = \frac{1+2+3}{3} = 2, \quad c_2 = \frac{10+11+12}{3} = 11.$
Iteration 2, assign: the same groups again, so nothing changes. **Converged:** clusters

$\{1, 2, 3\}$ and $\{10, 11, 12\}$.

Association rules: support, confidence, lift

For a rule $A \Rightarrow B$:

$$\text{support}(A \Rightarrow B) = \frac{\#\{A \cup B\}}{N}, \qquad \text{confidence} = \frac{\text{support}(A \cup B)}{\text{support}(A)}, \qquad \text{lift} = \frac{\text{confidence}}{\text{support}(B)}.$$

Worked Example: Bread ⇒ Butter

Out of $N = 100$ baskets: bread in 40, butter in 30, both in 20.
support $= 20/100 = 0.20$; confidence $= 20/40 = 0.50$; lift $= 0.50/0.30 \approx 1.67$.
Lift > 1 means buying bread makes butter *more* likely than chance (lift $= 1$ would mean independence).

2.3 Semi-Supervised Learning

Labelling data is expensive (a doctor must mark every scan), but unlabelled data is cheap. **Semi-supervised learning** uses a *small labelled set plus a large unlabelled set.*

- **Idea:** train on the few labels, predict labels for unlabelled data, then retrain using the confident predictions (*self-training*); or spread labels through similar points (*label propagation*).
- **Examples:** medical image classification, web-page classification, speech recognition.

2.4 Self-Supervised Learning

The model creates its **own labels from the data**. Hide part of the input and ask the model to predict it: the next word in a sentence, a masked patch of an image. No human labelling is needed, which is why this approach powers modern language models and large vision models.

2.5 Reinforcement Learning (RL)

In Plain Words

Training a dog: it tries actions, gets a treat for good behaviour and nothing (or a scolding) for bad behaviour. Over time it learns which actions earn the most treats. There is no answer key, only rewards.

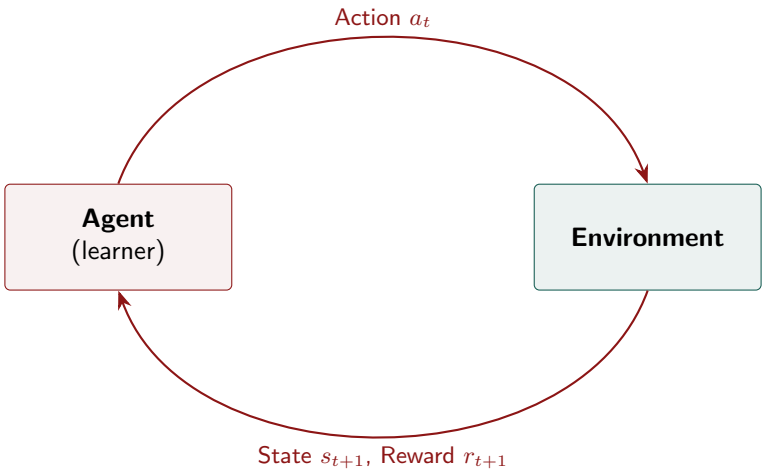


Figure 4. The reinforcement-learning loop.

Term	Meaning
Agent	The learner / decision maker
Environment	Everything the agent interacts with
State s	Current situation of the environment
Action a	A choice the agent can make
Reward r	Numerical feedback after an action
Policy π	The agent's strategy: which action to take in each state
Return	Total (discounted) future reward the agent tries to maximise

Exploration vs. exploitation: should the agent try new actions (explore) or keep using the best known one (exploit)? A common compromise is ε-*greedy*: explore with probability ε, otherwise exploit.

Q-learning update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

with learning rate α and discount factor γ.

Worked Example: One Q-learning update

$Q(s,a) = 0$, $\alpha = 0.5$, reward $r = 10$, $\gamma = 0.9$, $\max_{a'} Q(s',a') = 0$. $Q \leftarrow 0 + 0.5\,[10 + 0.9(0) - 0] = 5$.

Applications: game playing (Go, chess), robotics, self-driving, traffic-signal control, data-centre cooling, recommendation, algorithmic trading.

2.6 Comparison of the Learning Types

	Supervised	Unsupervised	Semi-supervised	Reinforcement
Data	Labelled	Unlabelled	Few labels + many unlabelled	No dataset; interaction
Goal	Predict output	Discover structure	Predict with fewer labels	Maximise cumulative reward
Feedback	Correct answer	None	Partial	Reward / penalty
Example	Spam detection	Customer segments	Medical image tagging	Game-playing agent
Typical algorithms	Regression, SVM, trees	k-Means, PCA	Self-training	Q-learning, policy gradient

2.7 Other Ways to Classify Learning Systems

Axis	Option A	Option B
Training mode	**Batch (offline):** trained on all data at once, then frozen	**Online (incremental):** updated continually as new data streams in
Generalisation style	**Instance-based:** compares new points with stored examples (k-NN)	**Model-based:** builds a model with parameters (linear regression)
Model form	**Parametric:** fixed number of parameters (linear/logistic regression)	**Non-parametric:** complexity grows with data (k-NN, decision trees)
What it models	**Discriminative:** learns $P(y \mid x)$ directly (logistic regression)	**Generative:** learns $P(x, y)$ and can create data (Naive Bayes, GANs)

Exam Tip

Typical question: "Differentiate between supervised and unsupervised learning." Answer in a table with at least five rows: data, goal, feedback, examples, algorithms. Draw one tiny diagram if time permits.

Pro Corner (for professionals)

Why labels matter mathematically. In supervised learning we minimise a loss $L(f(x), y)$ that explicitly uses y. Unsupervised methods optimise label-free objectives instead: k-Means minimises within-cluster variance $\sum_j \sum_{x \in C_j} \|x - \mu_j\|^2$; PCA maximises variance captured by a lower-dimensional projection; density models maximise likelihood $\prod_i p(x_i; \theta)$. Self-supervised learning blurs the line by deriving y from x itself (a pretext task), then transferring the learned representation to downstream supervised tasks.

3 Applications of Machine Learning

ML is already woven into daily life, often invisibly. The table below connects each **application** to the **learning type** and the **task** behind it, which is exactly how examiners like to see it.

Domain	Application	Learning type	Task
Healthcare	Detecting tumours in X-rays and scans; predicting disease risk; drug discovery	Supervised	Classification
Finance	Credit scoring; credit-card and UPI fraud detection; algorithmic trading	Supervised, unsupervised	Classification, anomaly detection
E-commerce	Product and movie recommendations; dynamic pricing	Unsupervised, supervised	Recommendation, regression
Transport	Self-driving cars; ETA and traffic prediction; route optimisation	Supervised, RL	Detection, control
Agriculture	Crop-yield prediction; plant-disease detection from leaf photos; irrigation advice	Supervised	Regression, classification
Education	Personalised learning paths; plagiarism detection; dropout prediction	Supervised	Classification
Manufacturin	Predictive maintenance; defect detection on assembly lines	Supervised, anomaly detection	Classification
Language (NLP)	Machine translation; chatbots; sentiment analysis; spam filtering	Supervised, self-supervised	Sequence modelling, classification
Vision	Face recognition; OCR; medical imaging; surveillance	Supervised	Detection, classification
Cybersecurity	Intrusion and malware detection; phishing detection	Supervised, unsupervised	Classification, anomaly detection
Marketing	Customer segmentation; churn prediction	Unsupervised, supervised	Clustering, classification
Smart cities / energy	Load forecasting; smart-grid balancing; waste management	Supervised, RL	Regression, control

3.1 Four Mini Case Studies

1. Email spam detection

- **Type:** supervised binary classification.
- **Data:** emails labelled spam / not spam.
- **Features:** word frequencies, presence of links, sender reputation, number of capital letters.
- **Algorithms:** Naive Bayes, logistic regression, SVM.
- **Metric:** precision matters (do not push genuine mail into spam) as well as recall.

2. Recommendation system

- **Type:** mostly unsupervised / hybrid.
- **Data:** users' past ratings, views and purchases.
- **Approaches:** *collaborative filtering* (users with similar taste like similar items), *content-based* (suggest items similar to what you liked), or hybrids.

- **Challenge:** the cold-start problem for new users or items with no history.

3. Credit-card fraud detection

- **Type:** supervised classification or unsupervised anomaly detection.
- **Data:** transactions (amount, time, location, merchant).
- **Challenge: extreme class imbalance**: fraud may be under 0.2% of transactions, so plain accuracy is misleading (see Chapter 4).

4. Medical diagnosis from images

- **Type:** supervised classification with deep learning.
- **Data:** scans labelled by expert radiologists.
- **Challenge:** labels are expensive (use semi-supervised or transfer learning); false negatives can be life-threatening, so recall is critical; the model must be explainable to doctors.

3.2 Responsible Use: Ethical Issues

Issue	What can go wrong	Mitigation
Bias and fairness	A hiring model trained on past biased decisions repeats them	Audit data and outcomes by group; rebalance; fairness metrics
Privacy	Models can leak or misuse personal data	Anonymisation, consent, differential privacy
Explainability	Users cannot understand why a loan was refused	Interpretable models, explanation tools
Accountability	Who is responsible for a wrong automated decision?	Human oversight, documentation, regulation
Security	Adversarial inputs can fool a model	Robust training, monitoring

Exam Tip

For "Explain any five applications of ML", pick five different domains, and for each write: *what the system does*, *type of learning*, and *one data example*. This three-part pattern earns more marks than a simple list.

Pro Corner (for professionals)

Choosing the metric is part of the application. In fraud (prevalence $\approx 0.1\%$), a model that labels everything "legitimate" scores 99.9% accuracy yet catches zero fraud. Use precision–recall curves (PR-AUC), cost-sensitive evaluation (cost of a missed fraud vs. cost of a false alarm) and threshold tuning. In regulated domains (credit, healthcare) add calibration, fairness constraints and model documentation (model cards).

4 The Machine Learning Workflow

A real ML project is far more than "run an algorithm". Professionals follow a repeatable pipeline. Most of the effort (often 60–80%) goes into data work, not modelling.

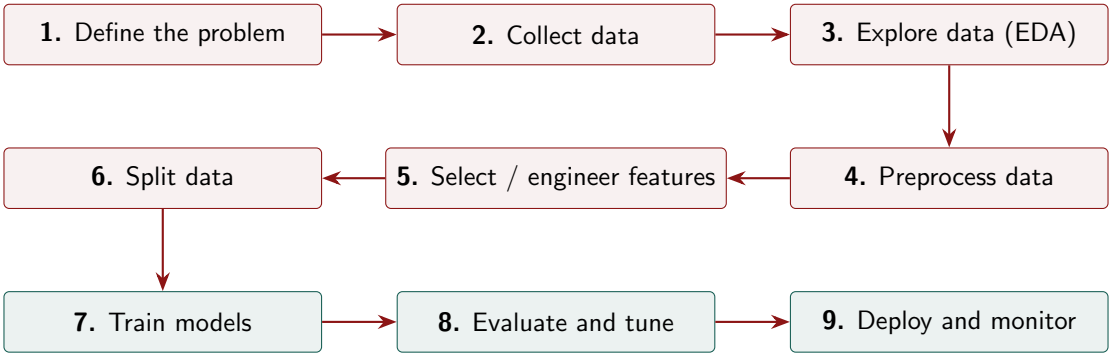


Figure 5. The ML workflow. In practice it is iterative: evaluation results send you back to earlier steps.

4.1 Step-by-Step Explanation

Step 1: Define the problem

Translate the business question into an ML task. "Reduce customer loss" becomes "predict which customers will leave in the next 30 days" (binary classification). Decide the success metric and what data is available.

Step 2: Collect data

Sources: databases, files (CSV, JSON), web scraping, APIs, sensors/IoT, surveys, public datasets (UCI, Kaggle). Ensure the data is *representative* of the real world where the model will run, and that you have the right to use it.

Step 3: Explore the data (EDA)

Look before you leap: shape, data types, summary statistics, distributions (histograms), relationships (scatter plots, correlation matrix), missing values, outliers, class balance.

Step 4: Preprocess the data

Clean errors, handle missing values and outliers, encode categories, scale numbers (Chapters 6 and 7).

Step 5: Feature selection and engineering

Create informative features (e.g. "age" from date of birth, "price per sq. ft.") and drop useless or redundant ones (Chapter 8).

Step 6: Split the data

Separate training, validation and test sets *before* fitting anything (Chapter 9).

Step 7: Choose and train models

Start with a simple baseline (e.g. logistic regression), then try stronger models. The training algorithm adjusts parameters to minimise the loss on the training data.

Step 8: Evaluate and tune

Measure performance on validation data using suitable metrics. Tune hyperparameters with grid search or random search; use cross-validation. Only at the very end, evaluate once on the test set.

Step 9: Deploy and monitor

Put the model into production (web service, mobile app, batch job). Monitor accuracy over time, since data distributions drift. Retrain when performance degrades. (This lifecycle practice is called **MLOps**.)

Tip

Industry frameworks describing the same loop are **CRISP-DM** (Business understanding → Data understanding → Data preparation → Modelling → Evaluation → Deployment) and **KDD**. Mentioning CRISP-DM in an answer shows depth.

4.2 Evaluation Basics You Must Know

Confusion matrix for binary classification

		Predicted: Positive	Predicted: Negative
Actual	Positive	TP (true positive)	FN (false negative)
	Negative	FP (false positive)	TN (true negative)

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \qquad \text{Precision} = \frac{TP}{TP+FP},$$

$$\text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2\,P\,R}{P+R}.$$

Worked Example: Metrics from a confusion matrix

$TP = 40,\ FP = 10,\ FN = 20,\ TN = 130$ (total 200).
Accuracy $= \frac{40+130}{200} = 0.85$. Precision $= \frac{40}{50} = 0.80$. Recall $= \frac{40}{60} \approx 0.667$.
$F_1 = \frac{2(0.80)(0.667)}{0.80+0.667} \approx 0.727$.

Watch Out

Accuracy paradox: if 99% of emails are not spam, a useless filter that always says "not spam" is 99% accurate. For imbalanced data rely on precision, recall and F_1, not accuracy.

Precision answers: *of everything I flagged, how much was truly positive?* Important when false alarms are costly (spam filter).

Recall answers: *of all real positives, how many did I catch?* Important when misses are costly (cancer screening).

Regression metrics

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|, \qquad \text{MSE} = \frac{1}{n}\sum (y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}},$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}.$$

RMSE is in the same units as the target and penalises large errors more than MAE. $R^2 = 1$ is a perfect fit; $R^2 = 0$ means no better than always predicting the mean.

4.3 Overfitting, Underfitting and the Bias–Variance Trade-off

In Plain Words

A student who memorises last year's paper word for word (overfitting) panics when the questions change. A student who barely studied (underfitting) fails everything. The goal is the student who *understands the concepts* and can handle new questions.

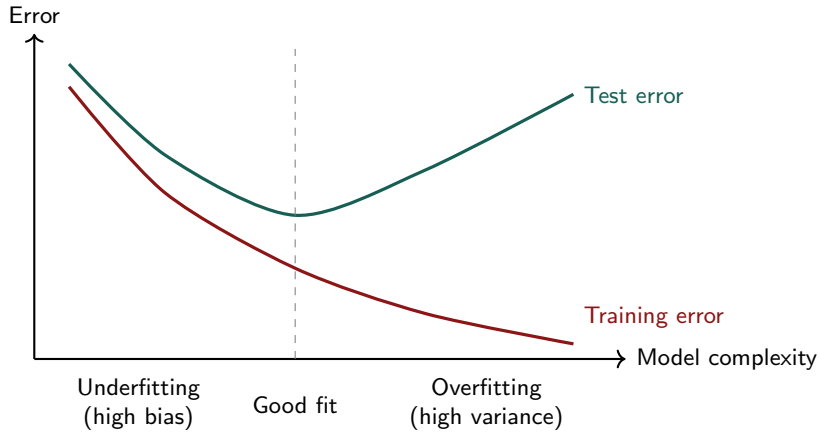


Figure 6. Training error keeps falling with complexity; test error falls and then rises.

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Test error	High	Low (close to training)	High (far above training)
Cause	Model too simple, too few features	Right complexity	Model too complex, too little data, noise memorised
Fixes	More complex model, better features, train longer	—	More data, simpler model, regularisation, early stopping, cross-validation, dropout

Pro Corner (for professionals)

Bias–variance decomposition (squared loss): $\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2}_{\text{wrong assumptions}} + \underbrace{\text{Variance}}_{\text{sensitivity to training set}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$. Making a model more flexible lowers bias but raises variance. **Regularisation** adds a penalty to the loss, for example ridge: $J(w) = \text{MSE} + \lambda \sum_j w_j^2$, trading a little bias for a large cut in variance.

4.4 Putting the Workflow in Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

df = pd.read_csv("customers.csv")            # 2. collect
print(df.info(), df.describe())               # 3. explore

X = df.drop(columns="churn")                  # features
y = df["churn"]                               # target
X_tr, X_te, y_tr, y_te = train_test_split(    # 6. split
    X, y, test_size=0.2, stratify=y, random_state=42)

num = ["age", "monthly_bill"]; cat = ["plan", "city"]
prep = ColumnTransformer([                    # 4. preprocess
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat)])

model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
model.fit(X_tr, y_tr)                         # 7. train
print(classification_report(y_te, model.predict(X_te)))   # 8. evaluate
```

Exam Tip

"Explain the ML workflow with a diagram" is a near-certain question. Draw the nine-box flow, add one line per box, and finish with a sentence on iteration and monitoring.

5 Data and Its Types

In Plain Words

Data is the *fuel* of machine learning. Just as a car runs differently on petrol, diesel or electricity, an ML algorithm needs to know *what kind* of data it is eating: numbers, categories, text or pictures. Knowing the type decides how we clean it, convert it and which algorithm we can use.

5.1 Anatomy of a Dataset

A tabular dataset is a grid: **rows are samples**, **columns are features**, and one special column is the **target** (label) in supervised learning.

Roll No	Gender	Attendance %	Grade	Study hrs/day	Result (target)
101	M	82.5	B	3.0	Pass
102	F	91.0	A	4.5	Pass
103	F	45.0	D	1.0	Fail
104	M	68.0	C	2.0	Pass

Here each row is one student (a sample), the middle four columns are features and "Result" is the target. "Roll No" is only an identifier: it carries no predictive meaning and is normally **dropped** before training.

5.2 Classification by Structure

Kind	Description	Examples
Structured	Fixed rows and columns with a schema; easy to store in relational databases	SQL tables, CSV, Excel sheets
Semi-structured	Has tags or keys but no rigid table	JSON, XML, HTML, log files
Unstructured	No predefined format; most real-world data (about 80%)	Text, images, audio, video, emails

Unstructured data must first be converted into numbers (for text: bag-of-words, TF-IDF, embeddings; for images: pixel arrays or learned features) before most algorithms can use it.

5.3 Classification by the Nature of the Attribute

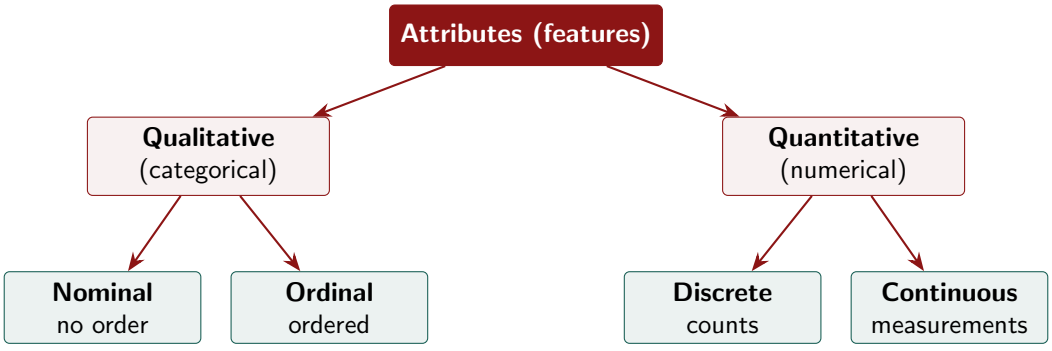


Figure 7. Taxonomy of attribute types.

Qualitative (categorical) data

- **Nominal:** categories with *no natural order*. Examples: blood group, city, colour, gender. Only equality is meaningful ($=$, $\neq$). Summary: mode.
- **Ordinal:** categories with a *meaningful order* but unequal or unknown gaps. Examples: grade (A $>$ B $>$ C), education level, satisfaction (poor / average / good). Summary: median and mode.
- **Binary:** just two categories (yes/no, 0/1). *Symmetric* if both outcomes matter equally (gender); *asymmetric* if one outcome is rare and more important (disease present/absent).

Quantitative (numerical) data

- **Discrete:** countable, usually whole numbers. Examples: number of children, number of defects, goals scored.
- **Continuous:** any value in a range, with decimals. Examples: height, weight, temperature, time.

5.4 Stevens' Four Levels of Measurement

Scale	Order?	Equal gaps?	True zero?	Examples
Nominal	No	No	No	Gender, city, blood group
Ordinal	Yes	No	No	Grades, rankings, star ratings
Interval	Yes	Yes	No	Temperature in °C, calendar year, IQ
Ratio	Yes	Yes	Yes	Height, weight, age, income, distance

Key Idea

Interval vs. ratio: 40°C is *not* "twice as hot" as 20°C because 0°C is arbitrary (no true zero). But 40 kg *is* twice 20 kg because 0 kg means "no weight". Ratios are meaningful only on a ratio scale.

Worked Example: Classify each attribute

Attribute	Type	Reason
Mobile number	Nominal (an identifier)	Arithmetic on it is meaningless
T-shirt size (S, M, L, XL)	Ordinal	Ordered, gaps unknown
Number of siblings	Discrete, ratio	Countable, true zero
Body temperature in °F	Continuous, interval	Equal gaps, no true zero
Annual salary	Continuous, ratio	True zero exists
Pass / Fail	Binary	Two outcomes

5.5 Other Kinds of Data You Will Meet

Data kind	Key property	Typical ML tasks
Time series	Observations ordered in time; neighbouring values are correlated	Forecasting, anomaly detection
Text	Variable-length sequences of words	Sentiment analysis, translation, chatbots
Images / video	Grids of pixels (and frames)	Classification, detection, segmentation
Audio / speech	Waveforms sampled over time	Speech recognition, music tagging
Graph / network	Nodes and relationships	Social-network analysis, fraud rings
Spatial / geo	Location coordinates	Route planning, crop mapping
Transactional	Sets of items per event	Market-basket analysis

5.6 Data Formats and Sources

Formats: CSV/TSV, Excel, JSON, XML, SQL databases, Parquet (columnar and compressed), image/audio files. **Sources:** *primary* data you collect yourself (surveys, sensors, experiments) versus *secondary* data collected by others (government portals, Kaggle, UCI repository, company logs).

5.7 Data Quality and Big Data

Good data is **accurate, complete, consistent, timely, unique (no duplicates) and valid**. Large-scale data is often described by the **5 V's**:

V	Meaning
Volume	Sheer size (terabytes to petabytes)
Velocity	Speed at which data arrives (streams)
Variety	Different forms: structured, semi-structured, unstructured
Veracity	Trustworthiness and noise level
Value	Usefulness of insight extracted

5.8 Why Data Type Drives Everything Next

Type	Typical preprocessing	Compatible methods
Continuous numeric	Scaling, outlier handling, transformations	Regression, SVM, k-NN, neural nets
Discrete numeric	Treat as numeric or as categories, depending on range	Most algorithms
Nominal	One-hot or target encoding	Trees, Naive Bayes, after encoding: most others
Ordinal	Ordinal encoding (preserve order)	Trees, regression, after encoding
Text	Tokenise, vectorise (TF-IDF, embeddings)	Naive Bayes, linear models, transformers
Image	Resize, normalise, augment	CNNs, vision transformers

Watch Out

Numbers do not always mean "numeric". A pin code or phone number is stored as digits but is a **nominal identifier**; feeding it directly into a regression would imply that pin code 700002 is "bigger" than 700001 in a meaningful way. Encode or drop such columns.

Pro Corner (for professionals)

In practice (pandas): `df.dtypes` gives `int64`, `float64`, `object`, `category`, `datetime64`. Convert text columns with a small set of values to `category` to save memory and signal intent. Watch for: *high-cardinality* categoricals (thousands of unique values, where one-hot explodes the width), *sparse* data (mostly zeros; use sparse matrices), *mixed types* in one column ("12", "N/A", "twelve"), and *data modalities* (multimodal models combine text + image + tabular).

Exam Tip

Expect: "Explain the different types of data/attributes with examples." Reproduce the taxonomy tree, then the four-row Stevens table, then one example each. Mention the interval-vs-ratio temperature example to stand out.

6 Data Preprocessing

In Plain Words

Before a chef cooks, vegetables are washed, peeled and chopped. Raw data is the same: it arrives dirty, inconsistent and in the wrong shape. **Preprocessing is the washing and chopping** that makes data ready for an algorithm. Skipping it is the single most common reason beginners get poor results.

Garbage in, garbage out: even the best algorithm cannot rescue bad data.

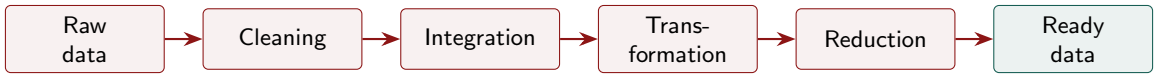


Figure 8. Major phases of data preprocessing.

6.1 Data Cleaning

Typical problems

- **Duplicates:** the same record repeated. Remove them, or they bias the model and leak between train and test sets.
- **Inconsistent entries:** "Kolkata", "kolkata", "Calcutta"; units mixed (kg and g); date formats mixed.
- **Invalid values:** age $= -5$, percentage $= 140$.
- **Noise:** random error in measurements.
- **Missing values** (Chapter 7) and **outliers** (below).

Smoothing noisy data by binning

Sort the values, split them into bins, then replace values in each bin by the bin mean, median or boundary.

Worked Example: Binning on prices $4, 8, 15, 21, 21, 24, 25, 28, 34$

Equal-frequency (depth 3) bins: $[4, 8, 15]$, $[21, 21, 24]$, $[25, 28, 34]$.
Smoothing by bin means: $[9, 9, 9]$, $[22, 22, 22]$, $[29, 29, 29]$ (e.g. $(4 + 8 + 15)/3 = 9$).
Smoothing by bin boundaries (each value moves to the closer of the bin's min or max): $[4, 4, 15]$, $[21, 21, 24]$, $[25, 25, 34]$. (In bin 1, 8 is 4 away from 4 and 7 away from 15, so it becomes 4.)

Outliers

An **outlier** is a value far from the rest. It may be an error (typing 1000 instead of 100) or a genuine rare event (a billionaire in a salary survey). Always investigate before deleting.

Detection methods

- **IQR rule:** $\text{IQR} = Q_3 - Q_1$; flag values below $Q_1 - 1.5\,\text{IQR}$ or above $Q_3 + 1.5\,\text{IQR}$ (box-plot whiskers).
- **Z-score rule:** flag $|z| > 3$ where $z = \frac{x-\mu}{\sigma}$ (works best for roughly bell-shaped data).
- **Model-based:** Isolation Forest, Local Outlier Factor, DBSCAN.
- **Visual:** box plots, scatter plots, histograms.

Worked Example: IQR outlier detection

Data: $5, 7, 8, 9, 10, 11, 12, 13, 100$ ($n = 9$).
Median $= 10$. Lower half $5, 7, 8, 9 \Rightarrow Q_1 = 7.5$. Upper half $11, 12, 13, 100 \Rightarrow Q_3 = 12.5$.
IQR $= 5$.
Fences: lower $= 7.5 - 1.5(5) = 0$; upper $= 12.5 + 7.5 = 20$. Since $100 > 20$, it is an **outlier**.

Treatment options: remove (if clearly an error); **cap / winsorise** (replace by the fence value); transform (log); impute; or keep and use a robust model (trees, median-based methods).

6.2 Data Integration

Combining data from several sources (databases, files, APIs) into one dataset. Issues: *schema matching* ("cust_id" vs "customer_no"), *entity resolution* (same person written differently), *conflicting values*, and *redundant attributes* (detect via correlation; drop one of two near-identical columns).

6.3 Data Transformation

Feature scaling

In Plain Words

Comparing a person's height in millimetres (1700) with their age in years (25) is unfair: the large number dominates. Scaling brings all features onto a comparable range so no feature shouts louder than others simply because of its units.

Why it matters: distance-based methods (k-NN, k-Means, SVM), gradient-based methods (linear/logistic regression, neural networks) and PCA are sensitive to scale. Tree-based models (decision tree, random forest, gradient boosting) are *not*.

Method	Formula	Result	Notes
Min–max normalisation	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$	Range $[0, 1]$	Simple; very sensitive to outliers
Standardisation (z-score)	$x' = \dfrac{x - \mu}{\sigma}$	Mean 0, std 1	Most common; good default
Robust scaling	$x' = \dfrac{x - \text{median}}{\text{IQR}}$	Centred on median	Handles outliers well
Max-abs scaling	$x' = \dfrac{x}{\max \lvert x \rvert}$	Range $[-1, 1]$	Keeps sparsity
Decimal scaling	$x' = \dfrac{x}{10^j}$	$\lvert x' \rvert < 1$	j = smallest integer making $\max \lvert x' \rvert < 1$
Unit-vector (L2)	$x' = \dfrac{x}{\lVert x \rVert_2}$	Row length 1	Used for text, cosine similarity

Worked Example: Min–max normalisation

Data: $10, 20, 30, 50, 90$. $x_{\min} = 10$, $x_{\max} = 90$, range $= 80$.
$10 \to 0$, $\quad 20 \to \frac{10}{80} = 0.125$, $\quad 30 \to 0.25$, $\quad 50 \to 0.5$, $\quad 90 \to 1$.

Worked Example: Z-score standardisation

Data: $2, 4, 4, 4, 5, 5, 7, 9$. Mean $\mu = \frac{40}{8} = 5$. Deviations squared: $9, 1, 1, 1, 0, 0, 4, 16$; sum $= 32$; population variance $= 4$; $\sigma = 2$.
z-values: $-1.5,\ -0.5,\ -0.5,\ -0.5,\ 0,\ 0,\ 1,\ 2$.

Worked Example: Decimal scaling

Values range from -300 to 986. The largest absolute value is 986; $j = 3$ makes $986/10^3 = 0.986 < 1$. So $986 \to 0.986$ and $-300 \to -0.3$.

Watch Out

Fit scalers on the training set only, then apply the same μ, σ (or min, max) to validation/test data. Computing them on the full dataset leaks test information into training (*data leakage*).

Non-linear transformations

For heavily skewed data (income, house prices) apply $\log(x)$ or $\log(1+x)$, square root, or power transforms (Box–Cox for positive data, Yeo–Johnson for any real values) to make the distribution closer to bell-shaped.

Encoding categorical variables

Most algorithms need numbers, so categories must be converted.

Method	How it works	When to use
Label / ordinal encoding	Map categories to integers (Low=0, Medium=1, High=2)	Ordinal data where order matters; tree models
One-hot encoding	One new 0/1 column per category	Nominal data with few categories
Dummy encoding	One-hot with one column dropped ($k-1$ columns)	Linear models (avoids redundancy)
Binary encoding	Category index written in binary across a few columns	High-cardinality nominal data
Frequency / count encoding	Replace category by how often it occurs	High cardinality, tree models
Target (mean) encoding	Replace category by mean of target for that category	High cardinality; needs care to avoid leakage

Worked Example: One-hot encoding

Colour $\in \{\text{Red}, \text{Green}, \text{Blue}\}$:

Colour	Is_Red	Is_Green	Is_Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

Watch Out

Label-encoding nominal data is dangerous for many models. Coding Red=0, Green=1, Blue=2 tells a linear model that Blue is "twice" Green. Use one-hot instead.

Dummy-variable trap: with an intercept, all k one-hot columns sum to 1, causing perfect multicollinearity; drop one column.

Discretisation (binning of continuous values)

Converting a numeric feature into intervals, e.g. age $\rightarrow$ {child, teen, adult, senior}.

- **Equal-width:** bins of the same size. For $4, 8, 15, 21, 21, 24, 25, 28, 34$ with 3 bins of width 10: $[4, 14) = \{4, 8\}$, $[14, 24) = \{15, 21, 21\}$, $[24, 34] = \{24, 25, 28, 34\}$.
- **Equal-frequency:** each bin has the same number of points (quantiles).
- **Custom / domain-based:** bins defined by experts (BMI categories).

6.4 Data Reduction

Reduce data volume while keeping its analytical value: **sampling** (use a representative subset), **dimensionality reduction** / feature selection (Chapter 8), **numerosity reduction** (histograms, clusters), **aggregation** (daily to monthly totals) and **compression**.

6.5 Handling Imbalanced Data

When one class is rare (fraud, disease), models ignore it. Remedies:

- **Resampling:** oversample the minority class (random duplication or **SMOTE**, which synthesises new minority points between neighbours); undersample the majority class.
- **Class weights:** penalise mistakes on the minority class more.
- **Better metrics:** precision, recall, F1, PR-AUC instead of accuracy.
- **Stratified splitting** so every split keeps the same class ratio.

Resample only the training set, never the test set.

6.6 Preventing Data Leakage with Pipelines

Data leakage means information from outside the training data (especially from the test set or the future) sneaks into model building, giving unrealistically good scores that collapse in production. The fix: split first, and wrap all preprocessing and the model in one `Pipeline` so every step is learned from training data only.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.svm import SVC

pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler",  StandardScaler()),
    ("model",   SVC())
])
pipe.fit(X_train, y_train)       # imputer + scaler learn ONLY from X_train
print(pipe.score(X_test, y_test))
```

6.7 Preprocessing Checklist

1. Inspect: shape, dtypes, summary statistics, class balance.

2. Remove duplicates; fix inconsistent labels, units and formats.
3. Handle missing values (Chapter 7) and outliers.
4. Drop identifiers and irrelevant columns.
5. Encode categorical variables.
6. Split into train/test (Chapter 9), then fit scalers and imputers on train only.
7. Scale numeric features if the algorithm needs it.
8. Select features or reduce dimensions (Chapter 8).
9. Handle class imbalance (training data only).

Exam Tip

Most-asked numerical: *apply min–max and z-score normalisation to a small list* and *smooth by bin means / boundaries.* Practise the three worked examples above until you can do them in two minutes each. Always show the formula first.

Pro Corner (for professionals)

Scaling and geometry. Euclidean distance $d(x, y) = \sqrt{\sum_j (x_j - y_j)^2}$ is dominated by the feature with the largest numeric range; standardisation equalises each axis variance, so k-NN, k-Means and RBF-SVMs treat features comparably. For gradient descent, unscaled features create elongated loss contours and slow, zig-zag convergence; scaling improves the condition number of the problem. **Target encoding** must be computed out-of-fold (or with smoothing) to prevent target leakage. For sparse text matrices avoid mean-centering (it destroys sparsity); use max-abs or L2 scaling.

7 Handling Missing Values

In Plain Words

Imagine a survey form where some people skipped a few questions. You now have blank boxes. You can throw away the incomplete forms, or you can make a sensible guess for the blanks. Handling missing values is deciding *which* of these to do, and doing it without fooling yourself.

7.1 Why Are Values Missing?

Equipment failure, people skipping questions, privacy (income left blank), data-entry errors, merging mismatched tables, or the value simply not applying ("spouse's name" for an unmarried person). Missing data appears as `NaN`, `None`, blank, "?", "N/A", or sentinel numbers like -999 or 0. **Always convert these to a proper missing marker first.**

7.2 Types of Missingness

Type	Meaning	Example
MCAR (Missing Completely At Random)	Missingness has no relation to any data, observed or not	A lab sample is dropped by accident
MAR (Missing At Random)	Missingness depends on *other observed* variables, not on the missing value itself	Men skip a "depression score" question more often; gender is known
MNAR (Missing Not At Random)	Missingness depends on the *missing value itself*	High earners refuse to disclose income

MCAR is the safest case (deleting rows does not bias results). MAR can be handled well by model-based imputation. MNAR is the hardest: the *fact* that a value is missing is itself informative, so add a missing-indicator feature and think about the data-collection process.

7.3 Step 1: Detect and Understand

```
df.isnull().sum()                    # count of missing per column
df.isnull().mean() * 100             # percentage missing per column
df.replace(["?", "N/A", -999], pd.NA, inplace=True)    # unify markers
```

Ask: *How much* is missing per column and per row? Is it *random* or patterned? Does the column even matter?

7.4 Step 2: Choose a Strategy

A. Deletion

- **Listwise (row) deletion:** drop any row with a missing value. Easy; fine if missingness is MCAR and only a few rows (say under 5%) are affected. Otherwise it wastes data and can bias results.
- **Column deletion:** drop a feature with too many missing values (a common rule of thumb is more than 40–60%), unless it is critical.

- **Pairwise deletion:** use all available values for each calculation (e.g. each correlation computed on rows complete for that pair). Keeps data but yields inconsistent sample sizes.

B. Simple imputation (filling)

Method	Use for	Caution
Mean	Numeric, roughly symmetric	Pulled by outliers; shrinks variance
Median	Numeric, skewed or with outliers	Ignores relationships between features
Mode (most frequent)	Categorical	May over-represent the majority class
Constant / "Unknown" category	Categorical where missingness is informative; or fill 0 when 0 is meaningful	Must not confuse with a real value
Forward / backward fill	Time series (carry last/next observation)	Poor if gaps are long
Interpolation (linear, spline)	Smooth time series	Assumes smooth trend

Worked Example: Mean vs. median imputation

Ages: $22, 25, \text{NaN}, 30, \text{NaN}, 28$. Known values: $22, 25, 30, 28$. Mean $= \frac{105}{4} = 26.25$. Median (sorted $22, 25, 28, 30$) $= \frac{25+28}{2} = 26.5$.
Salaries ('000): $30, 32, 35, \text{NaN}, 500$. Mean of known $= \frac{597}{4} = 149.25$, but median $= \frac{32+35}{2} = 33.5$. The outlier 500 drags the mean far above typical salaries, so the **median is the better fill** here.

C. Advanced imputation

- **k-NN imputation:** find the k most similar rows (using other features) and average (or vote on) their values.
- **Regression imputation:** predict the missing feature from the others with a regression model.
- **Iterative imputation (MICE):** repeatedly model each incomplete column from the rest, cycling until stable; can produce multiple imputed datasets to reflect uncertainty.
- **Tree-based (MissForest) imputation:** random-forest version of the above; handles mixed types and non-linearity.
- **Algorithms that tolerate missing values natively:** XGBoost, LightGBM, and recent scikit-learn tree models learn the best branch for missing entries.

Worked Example: k-NN imputation, $k = 2$

Rows (height cm, weight kg): $(165, 60)$, $(172, 70)$, $(180, 82)$, $(168, 62)$ and a new person $(170, ?)$.
Distance by height: $|170 - 165| = 5$, $|170 - 172| = 2$, $|170 - 180| = 10$, $|170 - 168| = 2$. The two nearest are heights 172 and 168.
Imputed weight $= \frac{70+62}{2} = 66\,\text{kg}$.

D. Missing-indicator feature

Add a 0/1 column "`income_was_missing`". This lets the model learn from the pattern of missingness (crucial for MNAR) while the original column is imputed.

7.5 Decision Guide

Situation	Reasonable action
Very few rows affected (under 5%), MCAR	Delete rows
Column mostly empty (over 50%) and not essential	Drop the column
Numeric, symmetric	Mean imputation
Numeric, skewed or outliers	Median imputation
Categorical	Mode or an "Unknown" category
Time series	Forward fill or interpolation
Features strongly related to each other	k-NN or iterative imputation
Missingness itself may carry meaning (MNAR)	Impute + add missing-indicator

7.6 Code

```
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer  # noqa
from sklearn.impute import IterativeImputer

imp_med = SimpleImputer(strategy="median", add_indicator=True)
imp_cat = SimpleImputer(strategy="most_frequent")
imp_knn = KNNImputer(n_neighbors=5)
imp_mice = IterativeImputer(max_iter=10, random_state=0)

X_train_imp = imp_med.fit_transform(X_train)   # learn median from TRAIN only
X_test_imp  = imp_med.transform(X_test)        # reuse the same medians

# pandas quick fixes
df["age"] = df["age"].fillna(df["age"].median())
df["city"] = df["city"].fillna("Unknown")
df = df.dropna(thresh=int(0.6 * df.shape[1]))  # keep rows with >=60% data
```

Watch Out

Common mistakes: (1) Imputing *before* splitting, so test data influences the fill value (leakage). (2) Filling with 0 when 0 is a real value. (3) Using the mean on skewed data. (4) Ignoring that imputation reduces natural variation and can distort correlations. (5) Not checking *why* data is missing.

Exam Tip

A 5-mark question usually asks: "List methods to handle missing values." Give: deletion (listwise, column), mean/median/mode imputation, constant fill, forward fill, regression/k-NN imputation, and mention MCAR/MAR/MNAR. For 10 marks add the mean-vs-median example and the decision table.

Pro Corner (for professionals)

Principled approaches. Under MAR, *multiple imputation* (Rubin's rules) draws m plausible completions, fits the analysis on each, and pools estimates $\bar{\theta} = \frac{1}{m}\sum \hat{\theta}_j$ with total variance $T = \bar{W} + (1 + \frac{1}{m})B$ (within- plus between-imputation variance), correctly reflecting imputation uncertainty, which single imputation understates. *Maximum likelihood via EM* estimates parameters directly from incomplete data. For MNAR, selection or pattern-mixture models and sensitivity analysis are needed because the data alone cannot identify the mechanism. In production, serialise the fitted imputer inside the model pipeline and monitor the rate of missingness as a drift signal.

8 Feature Selection

In Plain Words

Packing for a trip, you do not take your entire wardrobe; you take what you will actually use. Feature selection is **packing only the useful columns** for your model and leaving behind the irrelevant, repeated or noisy ones. A lighter bag is faster to carry and easier to manage.

Definition: Feature Selection

The process of choosing a subset of the original features that is most relevant for predicting the target, discarding the rest, while **keeping the features themselves unchanged**.

8.1 Why Select Features?

- **Reduce overfitting:** fewer irrelevant features means less noise to memorise.
- **Improve accuracy:** remove misleading features.
- **Faster training and prediction;** lower memory and storage.
- **Better interpretability:** 10 features are easier to explain than 1000.
- **Lower data-collection cost** in the future.
- **Beat the curse of dimensionality** (below).

The curse of dimensionality

As the number of features d grows, the space grows exponentially. To cover each axis with just 10 distinct values you need 10^d cells: 100 for $d = 2$, a million for $d = 6$. Data becomes *sparse*, distances between points become nearly equal, and models need exponentially more samples to generalise. Rule of thumb: keep features well below the number of samples, and prune aggressively when d is large.

8.2 Feature Selection vs. Extraction vs. Engineering

Concept	What it does	Example
Feature selection	*Keeps a subset* of existing features; originals unchanged	Keep "area" and "bedrooms", drop "seller_id"
Feature extraction	*Creates new, fewer features* as combinations of the old	PCA turns 50 correlated columns into 5 components
Feature engineering	*Constructs new meaningful features* from raw data using domain knowledge	Extract "day of week" from a timestamp; "BMI" from height and weight

8.3 Three Families of Selection Methods

	Filter	Wrapper	Embedded
Idea	Score each feature with a statistic, independent of any model	Search subsets, judged by actual model performance	Selection happens *during* model training
Speed	Very fast	Slow (many models trained)	Moderate
Considers feature interactions?	No (mostly univariate)	Yes	Yes
Overfitting risk	Low	Higher	Low to moderate
Examples	Variance threshold, correlation, chi-square, ANOVA F, mutual information	Forward selection, backward elimination, RFE	Lasso (L1), decision-tree / random-forest importance, elastic net

8.4 Filter Methods

Variance threshold

A feature that is (almost) constant carries no information. Remove features whose variance is below a threshold. For a binary feature that equals 1 with probability p, variance $= p(1-p)$; if $p = 0.95$ the variance is only 0.0475, so it is nearly constant.

Correlation (Pearson)

For two numeric variables,

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2}\,\sqrt{\sum (y_i - \bar{y})^2}}, \qquad -1 \le r \le 1.$$

- Feature vs. **target**: keep features with high $|r|$.
- Feature vs. **feature**: if $|r| > 0.9$ (multicollinearity) they are redundant; keep one, drop the other.

Worked Example: Pearson correlation by hand

$x = 1, 2, 3, 4, 5$ and $y = 2, 4, 5, 4, 5$. $\bar{x} = 3$, $\bar{y} = 4$.
$(x - \bar{x}) = -2, -1, 0, 1, 2$; $\quad (y - \bar{y}) = -2, 0, 1, 0, 1$.
$\sum (x-\bar{x})(y-\bar{y}) = 4+0+0+0+2 = 6$; $\quad \sum (x-\bar{x})^2 = 10$; $\quad \sum (y-\bar{y})^2 = 4+0+1+0+1 = 6$.
$r = \dfrac{6}{\sqrt{10}\sqrt{6}} = \dfrac{6}{\sqrt{60}} \approx \dfrac{6}{7.746} = 0.775$ (strong positive correlation).

Watch Out

Correlation measures only *linear* association and **does not imply causation**. A feature can have $r \approx 0$ with the target and still be strongly (non-linearly) related, or useful in combination with others.

Chi-square test (categorical feature vs. categorical target)

$$\chi^2 = \sum \frac{(O - E)^2}{E}, \qquad E = \frac{\text{row total} \times \text{column total}}{\text{grand total}}.$$

A large χ^2 means the feature and target are dependent, so the feature is useful.

Worked Example: Is "email contains a link" related to spam?

	Spam	Not spam	Total
Has link	30	10	40
No link	20	40	60
Total	50	50	100

Expected: has-link/spam $= \frac{40\times50}{100} = 20$; has-link/not-spam $= 20$; no-link/spam $= \frac{60\times50}{100} = 30$; no-link/not-spam $= 30$.
$\chi^2 = \frac{(30-20)^2}{20} + \frac{(10-20)^2}{20} + \frac{(20-30)^2}{30} + \frac{(40-30)^2}{30} = 5 + 5 + 3.33 + 3.33 = 16.67$.
With 1 degree of freedom the 5% critical value is 3.84. Since $16.67 > 3.84$, the feature is **significantly related** to spam: keep it.

ANOVA F-test and mutual information

- **ANOVA F-test:** numeric feature vs. categorical target; compares between-class variance to within-class variance (large F = useful).
- **Mutual information:** $I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$ measures how much knowing X reduces uncertainty about Y. It captures *non-linear* dependence and is zero only if X and Y are independent.
- **Information gain** (used in decision trees) is the reduction in entropy after splitting on a feature.

8.5 Wrapper Methods

Treat the model as a black box and search over feature subsets.

- **Forward selection:** start empty; repeatedly add the feature that improves performance the most; stop when no improvement.
- **Backward elimination:** start with all features; repeatedly remove the least useful one.
- **Stepwise (bidirectional):** add and remove as needed.
- **Recursive Feature Elimination (RFE):** train, rank features by importance/coefficient, drop the weakest, repeat until the desired number remains.

Cost: testing every subset of n features needs 2^n models ($n = 10 \Rightarrow 1024$; $n = 20 \Rightarrow 1{,}048{,}576$), which is infeasible, so greedy forward/backward search (about $\frac{n(n+1)}{2}$ models) is used instead.

Worked Example: Forward selection with accuracy as the score

Features A, B, C.
Round 1 (one feature): $A = 70\%$, $B = 75\%$, $C = 65\% \Rightarrow$ pick B.
Round 2 (add one to B): $\{B, A\} = 82\%$, $\{B, C\} = 78\% \Rightarrow$ pick A.
Round 3: $\{B, A, C\} = 81\% < 82\%$: no improvement, **stop**. Selected set: $\{A, B\}$.

8.6 Embedded Methods

- **Lasso (L1) regression:** minimise $\sum(y - \hat{y})^2 + \lambda \sum_j |w_j|$. The L1 penalty drives weak features' coefficients to *exactly zero*, which removes them automatically.
- **Ridge (L2):** penalty $\lambda \sum w_j^2$ shrinks coefficients but rarely makes them zero, so it is not a selector.
- **Elastic Net:** mixes L1 and L2; stable when features are correlated.

- **Tree-based importance:** decision trees and random forests report how much each feature reduces impurity (Gini/entropy) across splits.

8.7 Feature Extraction: a First Look at PCA

Principal Component Analysis builds new, uncorrelated features (principal components) ordered by how much variance they capture.

1. Standardise the features.
2. Compute the covariance matrix.
3. Find its eigenvalues and eigenvectors (eigenvectors are the component directions).
4. Sort by eigenvalue; keep the top k so that the cumulative explained variance $\frac{\sum_{i \le k} \lambda_i}{\sum_i \lambda_i}$ is, say, at least 95%.
5. Project the data onto these k directions.

Other extraction methods: LDA (supervised, maximises class separation), autoencoders, t-SNE / UMAP (mainly for visualisation).

Tip

Selection keeps meaning; extraction trades it for compactness. If you must explain results to a doctor or manager ("income and age drive the decision"), prefer selection. If you only need speed and accuracy on high-dimensional data (images, genes), extraction is fine.

8.8 Practical Code

```
from sklearn.feature_selection import (VarianceThreshold, SelectKBest,
                                       f_classif, chi2, RFE, SelectFromModel)
from sklearn.linear_model import LogisticRegression, Lasso
from sklearn.ensemble import RandomForestClassifier

X1 = VarianceThreshold(threshold=0.01).fit_transform(X_train)

skb = SelectKBest(score_func=f_classif, k=10).fit(X_train, y_train)
X2 = skb.transform(X_train)                          # filter method

rfe = RFE(LogisticRegression(max_iter=1000), n_features_to_select=8)
rfe.fit(X_train, y_train)                            # wrapper method
print(rfe.support_)

sfm = SelectFromModel(Lasso(alpha=0.05)).fit(X_train, y_train)   # embedded
imp = RandomForestClassifier(n_estimators=200, random_state=0).fit(X_train, y_train)
print(imp.feature_importances_)

# correlation-based redundancy check
corr = X_train.corr().abs()
```

Watch Out

Select features inside cross-validation / a pipeline, using training data only. Selecting on the whole dataset first and then splitting leaks label information and makes results look better than they are. Also remember: univariate filters can miss features that are useful only in combination.

Exam Tip

Likely questions: (i) *Differentiate filter, wrapper and embedded methods* (use the table); (ii) *What is the curse of dimensionality?*; (iii) *Selection vs. extraction*; (iv) a small numerical on **correlation** or **chi-square**. Learn the two formulas above.

Pro Corner (for professionals)

Deeper notes. (1) *Relevance vs. redundancy:* mRMR (minimum redundancy, maximum relevance) picks features maximising $I(f;y) - \frac{1}{|S|}\sum_{g\in S} I(f;g)$. (2) *Stability:* under correlated features, Lasso arbitrarily picks one; use stability selection (subsample, count selection frequency) or elastic net. (3) *Permutation importance* (shuffle one column and measure the score drop) is model-agnostic and avoids impurity-based bias toward high-cardinality features. (4) *SHAP values* give per-prediction attribution. (5) With $p \gg n$ (genomics), apply selection strictly inside nested cross-validation to avoid selection bias.

9 Training and Testing Data

In Plain Words

A teacher gives you practice questions to *learn* from (training data). To find out whether you truly understood, the final exam must contain *new* questions you have never seen (test data). If the exam repeated the practice questions, a top score would only prove you memorised them. ML works the same way.

The goal of ML is generalisation: good performance on data the model has *never seen*. We therefore always hold back some data purely for judging the model.

9.1 The Three Datasets

Set	Purpose	Used when	Analogy
Training set	Fit the model's parameters	During learning	Practice questions
Validation set	Tune hyperparameters, compare models, early stopping	During model development, many times	Mock test
Test set	Final, unbiased estimate of real-world performance	**Once**, at the very end	Final exam



Figure 9. A typical 60/20/20 split (other common choices: 80/20 or 70/15/15).

How big should each split be?

- **Small data (hundreds to thousands):** 70/30 or 80/20, plus cross-validation.
- **Large data (millions):** even 98/1/1 is enough since 1% is still tens of thousands of samples.
- The test set must be *large enough* to give a stable estimate and *representative* of future data.

Worked Example: Counting a split

Dataset: 1000 samples, split 80/20 $\Rightarrow$ 800 train, 200 test.
Class balance: 10% positives, i.e. 100 positives overall. A **stratified** split keeps the ratio: train has 80 positives out of 800, test has 20 out of 200. A plain random split might put, say, 30 positives in test and only 70 in train, distorting evaluation.

9.2 Splitting Strategies

Hold-out method

Split once (e.g. 80/20). Fast and simple, but the score depends on *which* rows landed in the test set, so with small data it is noisy. Always fix a `random_state` for reproducibility.

Stratified split

Preserves the class proportions in every subset. Use it for classification, especially with imbalanced classes.

k-fold cross-validation

Split the data into k equal folds. Repeat k times: train on $k-1$ folds, validate on the remaining one. Average the k scores. Every sample is used for both training and validation, giving a more reliable estimate. Common choice: $k = 5$ or 10.

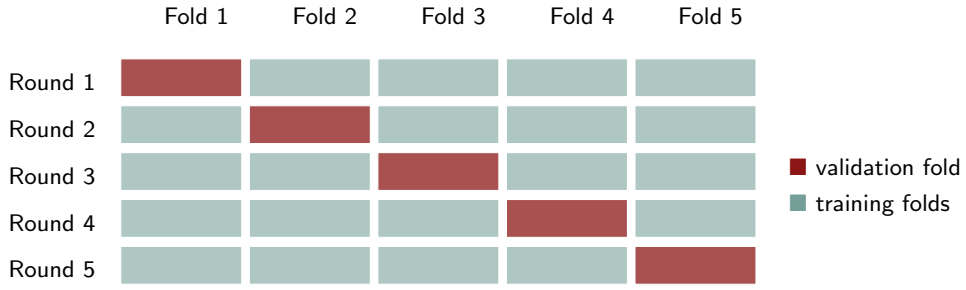


Figure 10. 5-fold cross-validation: each fold is the validation set exactly once.

Worked Example: Reading 5-fold results

100 samples, $k = 5 \Rightarrow$ each fold has 20 samples; each round trains on 80 and validates on 20.
Fold accuracies: 0.82, 0.85, 0.80, 0.88, 0.85.
Mean $= \frac{4.20}{5} = 0.84$. Deviations: -0.02, 0.01, -0.04, 0.04, 0.01; squares sum to 0.0038.
Sample standard deviation $= \sqrt{0.0038/4} \approx 0.031$.
Report: **accuracy** $= 0.84 \pm 0.03$. The $\pm$ shows how stable the model is across different data slices.

Other schemes

Method	Description	Use when
Stratified k-fold	Each fold keeps class ratios	Classification, especially imbalanced
Leave-one-out (LOOCV)	$k = n$: validate on a single sample each round	Very small datasets (costly otherwise)
Repeated k-fold	Repeat k-fold with different shuffles	Need lower-variance estimate
Group k-fold	Samples from the same group (patient, user) never split across train and validation	Repeated measurements per entity
Time-series split	Train on the past, validate on the future (expanding window)	Temporal data: **never shuffle**
Bootstrap	Sample n points *with replacement* for training; unused (out-of-bag) points for testing	Small data, ensembles (random forest)

In a bootstrap sample of size n, each point is left out with probability $(1 - \frac{1}{n})^n \to e^{-1} \approx 0.368$, so about **63.2%** of distinct points appear in training and **36.8%** are out-of-bag.

9.3 Diagnosing Models from Train and Test Scores

Train score	Test score	Diagnosis	Action
Low	Low	Underfitting (high bias)	Richer model, more features, less regularisation
High	Much lower	Overfitting (high variance)	More data, simpler model, regularise, feature selection
High	High, close to train	Good generalisation	Ship it (after final checks)
High	Suspiciously perfect	Possible data leakage	Audit the pipeline and features

9.4 Data Leakage: The Silent Killer

Leakage happens when information the model should not have at prediction time influences training.

- **Train–test contamination:** scaling, imputing or selecting features on the full dataset before splitting; duplicates appearing in both sets.
- **Target leakage:** a feature that is effectively the answer, e.g. "treatment given" when predicting "has disease", or "refund issued" when predicting fraud.
- **Temporal leakage:** using future data to predict the past (random shuffle on time series).
- **Test-set tuning:** repeatedly adjusting the model until the *test* score looks good, which turns the test set into a second validation set.

Key Idea

Golden rules. (1) Split first. (2) Learn every preprocessing step from training data only. (3) Use the validation set (or cross-validation) for all decisions. (4) Touch the test set once, at the end. (5) Keep the test set representative of production data.

9.5 Code

```
from sklearn.model_selection import (train_test_split, cross_val_score,
                                     StratifiedKFold, GridSearchCV, TimeSeriesSplit)
from sklearn.ensemble import RandomForestClassifier

# 60/20/20 split via two calls
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)  # 0.25 of 80% = 20%

# 5-fold stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(RandomForestClassifier(random_state=0),
                         X_train, y_train, cv=cv, scoring="f1")
print(scores.mean(), scores.std())

# hyper-parameter search with cross-validation
grid = GridSearchCV(RandomForestClassifier(random_state=0),
                    {"n_estimators": [100, 300], "max_depth": [None, 8]},
                    cv=cv, scoring="f1")
grid.fit(X_train, y_train)
print(grid.best_params_, grid.score(X_test, y_test))   # test used once

# time series: past -> future only
```

```
tscv = TimeSeriesSplit(n_splits=5)
```

Exam Tip

Be ready for: "Why do we split data into training and testing sets?", "Explain k-fold cross-validation with a diagram" and "What is data leakage?" The 60/20/20 bar diagram and the 5-fold grid above are quick to draw and score well.

Pro Corner (for professionals)

Nested cross-validation: an inner loop selects hyperparameters while an outer loop estimates generalisation, preventing the optimistic bias that appears when the same folds are used to both tune and report. **Distribution shift:** the i.i.d. assumption (train and test drawn from the same distribution) often fails in production; types are covariate shift ($P(x)$ changes), label shift ($P(y)$ changes) and concept drift ($P(y \mid x)$ changes). Mitigate with temporally-aware validation, monitoring and scheduled retraining. **Learning curves** (score vs. training-set size) show whether more data will help: a persistent gap means variance (get more data), while converged low scores mean bias (change the model).

10 Revision Toolkit

10.1 One-Page Summary

Topic	Remember this
What is ML	Learning patterns from data to predict on new data. Mitchell: improve at task T, measure P, with experience E. Traditional: data + rules → answers; ML: data + answers → rules.
Types	Supervised (labelled: classification, regression); Unsupervised (clustering, dimensionality reduction, association); Semi- and self-supervised; Reinforcement (agent, reward, policy).
Applications	Healthcare, finance, e-commerce, transport, agriculture, NLP, vision, security. Always name the learning type and task.
Workflow	Problem → collect → EDA → preprocess → features → split → train → evaluate/tune → deploy/monitor.
Data types	Structured / semi / unstructured. Nominal, ordinal, discrete, continuous. Stevens: nominal, ordinal, interval, ratio.
Preprocessing	Clean (duplicates, noise, outliers), integrate, transform (scale, encode, bin), reduce. Fit on train only.
Missing values	MCAR / MAR / MNAR. Delete, or impute: mean (symmetric), median (skewed), mode (categorical), k-NN, MICE; add missing-indicator.
Feature selection	Filter (fast, statistics), wrapper (search with model), embedded (Lasso, trees). Selection keeps features; extraction (PCA) creates new ones.
Train / test	Train fits, validation tunes, test judges once. Hold-out, stratified, k-fold, LOOCV, time-series split. Avoid leakage.

10.2 Formula Sheet

Name	Formula
Min–max	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$
Z-score	$z = \dfrac{x - \mu}{\sigma}$
Decimal scaling	$x' = \dfrac{x}{10^j}$, smallest j with $\max \lvert x' \rvert < 1$
IQR fences	$Q_1 - 1.5\,\mathrm{IQR}$ and $Q_3 + 1.5\,\mathrm{IQR}$, with $\mathrm{IQR} = Q_3 - Q_1$
Accuracy / Precision / Recall / F1	$\frac{TP+TN}{N}$, $\frac{TP}{TP+FP}$, $\frac{TP}{TP+FN}$, $\frac{2PR}{P+R}$
MSE / RMSE / MAE / R^2	$\frac{1}{n}\sum(y-\hat{y})^2$, $\sqrt{\mathrm{MSE}}$, $\frac{1}{n}\sum\lvert y-\hat{y}\rvert$, $1 - \frac{SS_{res}}{SS_{tot}}$
Gradient descent	$\theta \leftarrow \theta - \alpha \nabla J(\theta)$
Support / confidence / lift	$\frac{\#(A \cup B)}{N}$, $\frac{\mathrm{supp}(A \cup B)}{\mathrm{supp}(A)}$, $\frac{\mathrm{conf}}{\mathrm{supp}(B)}$
Pearson r	$\dfrac{\sum(x-\bar{x})(y-\bar{y})}{\sqrt{\sum(x-\bar{x})^2 \sum(y-\bar{y})^2}}$
Chi-square	$\sum \frac{(O-E)^2}{E}$, $\quad E = \frac{\mathrm{row} \times \mathrm{col}}{\mathrm{total}}$
Lasso / Ridge	$\mathrm{MSE} + \lambda \sum \lvert w_j \rvert$ / $\mathrm{MSE} + \lambda \sum w_j^2$
Q-learning	$Q \leftarrow Q + \alpha[r + \gamma \max Q' - Q]$

10.3 Commonly Confused Pairs

Pair	First	Second
Classification vs regression	Predicts a category	Predicts a number
Classification vs clustering	Labels known (supervised)	Groups discovered (unsupervised)
Parameter vs hyperparameter	Learned from data	Set before training
Normalisation vs standardisation	Rescale to a range, usually $[0, 1]$	Mean 0, standard deviation 1
Feature selection vs extraction	Choose a subset	Build new combined features
Validation vs test set	For tuning, used often	For the final report, used once
Precision vs recall	Correct among predicted positives	Found among actual positives
Bias vs variance	Error from too-simple assumptions	Error from sensitivity to the training sample
Overfitting vs underfitting	Memorises training data	Too simple to learn the pattern
Label vs one-hot encoding	Integers, implies order	0/1 columns, no order

10.4 Short-Answer Practice (with model answers)

1. **Define machine learning.** Learning from data to improve performance on a task without explicit programming; formally, improving performance P on task T with experience E.
2. **Difference between AI and ML?** AI is the broad goal of intelligent machines; ML is a subset that achieves it by learning from data.
3. **What are classification and regression?** Supervised tasks predicting discrete categories and continuous values respectively.
4. **Give two clustering applications.** Customer segmentation; grouping news articles by topic.
5. **What is reinforcement learning?** An agent learns a policy by interacting with an environment and maximising cumulative reward.
6. **What is a feature?** A measurable property of a sample used as model input.
7. **Nominal vs ordinal data?** Nominal categories have no order (blood group); ordinal categories are ordered (grade A, B, C).
8. **Why preprocess data?** Raw data is incomplete, noisy and inconsistent; models need clean, numeric, comparable inputs.
9. **Name three ways to handle missing values.** Delete rows or columns; fill with mean, median or mode; model-based imputation such as k-NN or MICE.
10. **When is median better than mean for imputation?** When the data is skewed or has outliers.
11. **What is an outlier and one way to detect it?** A value far from the rest; flag values outside $Q_1 - 1.5\,\mathrm{IQR}$ or $Q_3 + 1.5\,\mathrm{IQR}$.
12. **Why scale features?** So that large-range features do not dominate distance- or gradient-based algorithms.

13. **What is feature selection?** Choosing the most relevant subset of features to improve accuracy, speed and interpretability.
14. **Name the three families of feature selection.** Filter, wrapper, embedded.
15. **Why a separate test set?** To get an unbiased estimate of performance on unseen data.
16. **What is overfitting?** Model fits training data (including noise) too closely and performs poorly on new data.

10.5 Long-Answer Outlines

Question	Skeleton that earns full marks
Explain the types of machine learning	Definition; tree diagram; each type with data, goal, algorithms, example; comparison table.
Explain the ML workflow	Nine-box diagram; one or two lines per step; iteration and monitoring; example (churn).
Explain data preprocessing in detail	Why; cleaning, integration, transformation, reduction; scaling formulas with a numeric; encoding; leakage warning.
Discuss ways to handle missing data	MCAR/MAR/MNAR; deletion; imputation methods; mean-vs-median example; decision table; pitfalls.
Explain feature selection methods	Need and curse of dimensionality; filter, wrapper, embedded table; one numeric (correlation or χ^2); extraction vs selection.
Explain training/testing data and cross-validation	Roles of three sets; split ratios; k-fold diagram; leakage; overfitting diagnosis.

10.6 Multiple-Choice Practice

1. Predicting tomorrow's temperature is an example of: (a) classification (b) regression (c) clustering (d) association
2. k-Means is a(n): (a) supervised (b) unsupervised (c) reinforcement (d) semi-supervised algorithm
3. Learning by reward and penalty describes: (a) supervised (b) unsupervised (c) reinforcement (d) self-training
4. Blood group is measured on which scale? (a) nominal (b) ordinal (c) interval (d) ratio
5. Temperature in °C is: (a) nominal (b) ordinal (c) interval (d) ratio
6. Min–max normalisation maps data into: (a) $[-1, 1]$ (b) $[0, 1]$ (c) mean 0 (d) unit length
7. Best simple imputation for a skewed numeric column: (a) mean (b) median (c) zero (d) drop column
8. Missingness depending on the missing value itself is: (a) MCAR (b) MAR (c) MNAR (d) none
9. Which is an embedded feature-selection method? (a) chi-square (b) forward selection (c) Lasso (d) correlation filter
10. RFE is a: (a) filter (b) wrapper (c) embedded (d) clustering method
11. Scaling is generally *unnecessary* for: (a) k-NN (b) SVM (c) decision trees (d) neural nets
12. The test set should be used: (a) every epoch (b) for tuning (c) once at the end (d) never
13. In 5-fold cross-validation on 100 samples, each training set has: (a) 20 (b) 50 (c) 80 (d) 100

14. Precision equals: (a) $\frac{TP}{TP+FN}$ (b) $\frac{TP}{TP+FP}$ (c) $\frac{TN}{TN+FP}$ (d) $\frac{TP+TN}{N}$
15. Very low training error but high test error indicates: (a) underfitting (b) overfitting (c) good fit (d) bias
16. The dummy-variable trap is avoided by using how many columns for k categories? (a) $k+1$ (b) k (c) $k-1$ (d) 1
17. A rule with lift > 1 shows: (a) negative association (b) independence (c) positive association (d) no support
18. Scalers should be fitted on: (a) the full data (b) test data (c) training data only (d) validation only

Tip

Answers: 1-b, 2-b, 3-c, 4-a, 5-c, 6-b, 7-b, 8-c, 9-c, 10-b, 11-c, 12-c, 13-c, 14-b, 15-b, 16-c, 17-c, 18-c.

10.7 Glossary

Term	Meaning
Algorithm	Step-by-step procedure used to learn a model
Bias (statistical)	Systematic error from overly simple assumptions
Class imbalance	One class far rarer than others
Cross-validation	Repeated train/validate splitting to estimate performance reliably
Curse of dimensionality	Problems caused by very many features relative to samples
Data leakage	Information from outside the training data influencing the model
EDA	Exploratory data analysis: summarising and visualising data first
Feature engineering	Creating informative features from raw data
Generalisation	Ability to perform on unseen data
Hyperparameter	Setting fixed before training (learning rate, k)
Imputation	Filling in missing values
Loss function	Measures the error the model tries to minimise
MICE	Multiple imputation by chained equations
Multicollinearity	Features that are highly correlated with each other
Noise	Random error or irrelevant variation in data
Normalisation	Rescaling features to a common range
One-hot encoding	Representing a category as a 0/1 vector
Outlier	Observation far from the bulk of the data
Overfitting	Fitting noise in training data; poor on new data
PCA	Method to compress features into uncorrelated components
Regularisation	Penalty that discourages overly complex models
SMOTE	Synthetic minority oversampling technique
Stratification	Keeping class proportions equal across splits
Underfitting	Model too simple to capture the pattern
Variance (model)	Sensitivity of the model to changes in the training sample

End of Module 1 — Introduction to Machine Learning
Course instructor: Prof. Rajesh Nag